

Faculty of Engineering & Technology
BGS Institute of Technology
(Constituent College of Adichunchanagiri University)
Department of Artificial Intelligence & Machine Learning



A Project Phase-II Report
on

**“An End-to-End Workflow for Enhancing Text Classification Models
with Generative AI-aided Data Augmentation ”**

Submitted in partial fulfillment for the award of degree of

Bachelor of Engineering
in
Artificial Intelligence & Machine Learning

Submitted by:

HITHASHREE H P	[22MAE018]
LAVANYAB N	[22AME024]
SUMA T	[22AME051]

Under the Guidance of:

Ms. Smitha K
Assistant Professor
Dept. of AI&ML
BGSIT, BG Nagara

Coordinator:

Mrs. Nischitha A R
Assistant Professor
Dept. of IS&E
BGSIT, BG Nagara



DEAPRTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
BGS INSTITUTE OF TECHNOLOGY
BG NAGAR-571448
2025-2026

Faculty of Engineering & Technology
BGS Institute of Technology
(Constituent College of Adichunchanagiri University)
Department of Artificial Intelligence & Machine Learning



CERTIFICATE

This is to certify that the project work phase-II entitled “**An End-to-End Workflow for Enhancing Text Classification Models with Generative AI-aided Data Augmentation**” is a bonafide work carried out by **Ms. Hithashree H P (22AME18)**, **Ms. Lavanya B N (22AME024)** and **Ms. Suma T (22AME051)** students of Faculty of Engineering and Technology, BGS Institute of Technology, BG Nagara in partial fulfilment for the award of Bachelor of Engineering in Artificial Intelligence & Machine Learning of Adichunchanagiri university during the academic year 2025-26. The project report has been approved as it satisfies the academic requirements with respect to the project work prescribed for bachelor of engineering degree.

Signature of Guide:

Ms. Smitha K
Assistant Professor
Dept. of AI&ML, BGSIT

Signature of Coordinator:

Mrs. Nischitha A R
Assistant Professor
Dept. of IS&E, BGSIT

Signature of HOD:

Dr. Siddartha B K
Professor & Head
Dept. of AI&ML, BGSIT

Signature of Principal:

Dr. B N Shobha
Dean&Principal
BGSIT

Viva -Voce

Internal Examiner

Name: _____
Signature: _____

External Examiner

Name: _____
Signature: _____

ACKNOWLEDGEMENT

We take this opportunity to thank a lot of eminent personalities, without whose constant encouragement, this endeavor of mine would not have become a reality.

At first, we thank **Adichunchanagiri University, B G Nagara**, for having this Project Work phase-II as part of its curriculum, which gave me a wonderful opportunity to work on my research and presentation abilities and **BGSIT** for providing me with such excellent facilities, without which this Project Work phase-II could not have acquired the shape it has now done.

We sincerely convey our regards and thanks to **Dr. B N Shobha, Principal and Dean, Faculty of Engineering and Technology, BGSIT, B G Nagara, Nagamangala Taluk, Mandya District**, for giving us a chance to carry out and present our Project Work phase-II.

Our sincere thanks to **Dr. Siddartha B K, Professor and Head, Department of AI&ML, BGSIT, B G Nagara, Nagamangala Taluk, Mandya District**, for giving us a chance to carry out and present our Project Work phase-II with all the support and facilities.

We would like to thank **Ms. Smitha K, Assistant Professor, Department of AI&ML, BGSIT, B G Nagara, Nagamangala Taluk, Mandya District**, our honorable guide who stood as an excellent guide to carry out our work has been always available as an expressive evaluator for the creation and correction of the report towards our work.

We are thankful to our project coordinator **Mrs. Nischitha A R, Assistant Professor, Department of IS&E**, for her support and encouragement.

Our heartfelt gratitude to all the teaching and non-teaching faculties of **Artificial Intelligence & Machine Learning Department, BGSIT, B G Nagara, Nagamangala Taluk, Mandya District**, for their support and guidance towards the completion of our Project Work phase-II.

Finally, we would also extend our heartfelt thanks to our family members, classmates, friends and well-wishers for their support and encouragement throughout this effort.

Hithashree H P	(22AME018)
Lavanay B N	(22AME024)
Suma T	(22AME051)

ABSTRACT

The text-classification performance by augmenting training corpora with external knowledge synthesized by a generative AI system. The objective is to mitigate long- tail class imbalance and out-of-distribution weaknesses without costly data collection. To accomplish this, the team generated topic-conditioned articles with ChatGPT and integrated them into the training loop of three architectures—CNN, HiSAN, and BERT—on the multi-label Reuters corpus. The study systematically varied augmentation factors (sample count, variability, and length) and evaluated effects using macro- and micro-averaged precision, recall, and F1. A head-to-head comparison with the widely used EDA (Easy Data Augmentation) baseline was also conducted.

The augmented pipelines yielded consistent gains, with the largest improvement observed for BERT. Variability mattered more than mere quantity: adding 30 distinct samples outperformed repeating small sets, and longer synthetic texts (≈ 150 – 256 words) modestly outperformed very short ones (≈ 50 words). Against EDA, generative augmentation delivered larger macro-F1 gains while converging in fewer epochs, and it uniquely supports zero-shot labels. These results suggest that generative AI-aided augmentation is a practical, low-overhead strategy to bolster minority- class performance and overall robustness in text classification, with clear applicability to other NLP domains facing data scarcity or distribution shift.

TABLE OF CONTENTS

CONTENT	PAGE NO
TITLE	
ACKNOWLEDGEMENT	
ABSTRACT	I
TABLE OF CONTENTS	li
LIST OF FIGURES	lii
LIST OF TABLES	Iv
CHAPTER 1: INTRODUCTION	1-2
CHAPTER 2: LITERATURE SURVEY	3-8
CHAPTER 3: PROBLEM STATEMENT	5-7
3.1 Existing System	5
3.2 Proposed System	6-7
CHAPTER 4: SYSTEM REQUIREMENT	8-12
4.1 Functional Requirement	8-9
4.2 Non Functional Requirement	10-11
4.3 Hardware Requirement	11
4.4 Software Requirement	12
CHAPTER 5: SYSTEM ANALYSIS	13-21
5.1 System Analysis	13-15
5.2 System Design	15-19
5.3 Data Base Design	19-20
5.4 Applications	21
CHAPTER 6: IMPLEMENTATIONS	22-25
CONCLUSION	26
REFERENCES	27

LIST OF FIGURES

Fig.no	Figure Name	Page. No
4.1	Use case Diagram	9
5.1	Data Flow Diagram-Level (0) Context	17 18
5.2	Data Flow Diagram-Level (1) High level Process	19
5.3	Data Flow Diagram-Level (2) Data augmentation Process	20
5.4	Entity Relationship Diagram	24
6.1	User input Diagram Output	25
6.2	Diagram	

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparison of Existing Papers	3
4.1	Hardware Requirements	11
4.2	Software Requirements	12

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The text-classification models can be improved by augmenting their corpora with external knowledge samples generated by a large generative model (ChatGPT). The rationale is to mitigate class imbalance and coverage gaps—common in multi-label news corpora such as Reuters by synthesizing label- conditioned articles that expose models to broader lexical and semantic variation. Concretely, the team evaluated three architectures (CNN, HiSAN, BERT) and systematically varied augmentation knobs sample count, variability (distinct vs. repeated samples), and article length while assessing macro and micro-averaged precision/recall/F1.

At a high level, the method (i) generates Reuters-style articles conditioned on each label, (ii) injects these synthetic texts into training splits, and (iii) measures performance deltas and convergence behavior across models and augmentation settings. The design emphasizes variability and length as potential levers (e.g., comparing 30 distinct samples to repeated sets; testing

~50/150/256-word targets) to understand whether richer, longer samples improve robustness more than simply adding more of the same.

Results indicate consistent improvements in macro-F1—most notably for BERT with distinct samples outperforming duplicated ones and longer texts modestly surpassing very short ones. Compared to EDA, generative augmentation yielded larger macro-F1 gains and faster convergence (BERT training

~13–16 epochs vs. ~30–35 with EDA), while uniquely supporting zero-shot labels because it is not constrained by the original dataset distribution. These findings suggest that carefully prompted, label- aware generation can bolster minority-class performance, reduce epoch budgets, and extend models to underrepresented or unseen labels.

The approach is practical and flexible but not without caveats: prompt quality materially affects sample quality, and generative models can at times introduce misleading content; thus, prompt engineering and minimal curation remain important. Overall, the project positions generative AI- aided augmentation as a cost-effective pathway to improve robustness and class balance in text classification and as a generalizable recipe for NLP tasks facing data scarcity or distribution shift.

The project seeks to answer whether generative AI-aided data augmentation can meaningfully improve text-classification performance—especially for under-represented (long-tail) labels—beyond what simple perturbation methods (e.g., EDA) or naïve oversampling provide. Multi-label corpora like Reuters are severely imbalanced (many topics have very few samples), which depresses accuracy for minority classes; merely adding duplicates can inflate sample size without adding genuine linguistic or topical diversity. The study therefore asks: do label-aware, externally informed synthetic texts (generated by a large language model) supply the kind of variability and richness that boosts macro-F1 and overall robustness, or are observed gains just a sample-count artifact.

1.2 OBJECTIVES

- To analyze how sample size, variability, and length of generated texts affect model performance.
- To evaluate the impact of generative AI-aided augmentation on text classification models (CNN, HiSAN, BERT).
- To design a graphical workflow representation that captures the integration of synthetic and original datasets for improved clarity.
- To incorporate Explainable AI techniques (e.g., attention visualization, SHAP/LIME) to improve transparency in how augmented data influences classification outcomes.
- To compare generative AI augmentation with EDA to establish benchmarks.

1.3 PROJECT SCOPE

This work is most valuable anywhere text classification suffers from class imbalance, sparse labels, or domain shift, and where collecting new labeled data is costly. By injecting label-aware, generative samples into training, the approach improves macro-F1 i.e., performance on minority classes without hurting overall micro-scores, making it well-suited for long-tail, multi-label tasks.

Core application areas

- News & media tagging (topics, beats, multi-label categories) where many labels have few examples; synthetic Reuters-style articles demonstrated sizable gains, especially for BERT.
- Regulatory/compliance labeling (e.g., policy clauses, risk categories) that require recall on rare classes; minority-label accuracy benefits most from EK augmentation.
- Healthcare, legal, and finance document triage where data access is constrained and rare categories matter (e.g., uncommon conditions or statutes); EK samples add linguistic variety beyond simple oversampling.

CHAPTER 2

LITERATURE SURVEY

A literature survey or a literature review in a project report shows the various analyses and research made in the field of interest and the results already published, taking into account the various parameters of the project and the extent of the project.

A literature survey includes the following

- Existing theories about the topic which are accepted universally.
- Books written on the topic, both generic and specific.
- Research done in the field usually in the order of oldest to latest.
- Challenges being faced and on-going work, if available.

Literature survey describes about the existing work on the given project. It deals with the problem associated with the existing system and also gives user a clear knowledge on how to deal with the existing problems and how to provide solution to the existing problems.

1. Easy Data Augmentation Techniques for Boosting Performance on Text Classification Tasks

J. Wei and K. Zou, 2023

This paper introduces simple yet effective augmentation techniques — synonym replacement, random insertion, random swap, and random deletion — to boost text classification performance without requiring large datasets or complex pipelines. These four operations collectively form the EDA framework, designed to increase training data diversity with minimal effort.

Advantages: The methods are extremely easy to implement and require no additional model training or external resources. They are computationally inexpensive, making them accessible even in low-resource settings. The techniques consistently improve classification performance across a range of NLP tasks and work well with small training datasets where data scarcity is a concern. The simplicity of the approach also means it can be combined easily with other augmentation strategies for further gains.

Disadvantages: The techniques rely on surface-level word manipulations and do not leverage any deep semantic understanding of the text. This means the augmented samples may lack the linguistic variety and naturalness that more advanced methods like back-translation or LLM-based generation can produce.

2. Text Generation for Imbalanced Text Classification *Akkaradamrongrat, S., Kachamas, P., and Sinthupinyo, 2022*

This study proposes using text generation models to address the problem of class imbalance in text classification by synthetically generating minority-class samples. The approach aims to balance the training distribution and improve classifier performance on underrepresented categories without the need for manual data collection.

Advantages: It effectively addresses class imbalance, which is a common and critical problem in real-world text classification tasks. By generating synthetic minority-class samples, it reduces the classifier's bias toward majority classes and improves overall model fairness and accuracy.

Disadvantages: The approach is heavily dependent on the quality and capability of the generative model being used. Weaker generative models may produce samples that are grammatically incorrect, contextually inappropriate, or repetitive.

3. Learning Data Manipulation for Augmentation and Weighting *Hu, Z., Tan, B., Salakhutdinov, R. R., Mitchell, T. M., and Xing, 2022*

This paper proposes a sophisticated meta-learning framework in which the model automatically learns optimal data augmentation and weighting strategies during training. Rather than relying on fixed, manually designed augmentation rules, the system dynamically adapts its approach based on validation feedback, making the augmentation process more intelligent and task-aware.

Advantages: By automating the augmentation and weighting process, this framework removes the burden of manually designing augmentation pipelines for each specific task. It adapts dynamically to the characteristics of the dataset, leading to more targeted and effective data manipulation. The approach has demonstrated strong performance improvements on multiple NLP benchmarks and is particularly useful in scenarios where the optimal augmentation strategy is not obvious.

Disadvantages: The framework is significantly more complex than simpler augmentation methods, making it harder to implement, debug, and interpret. The meta-learning process introduces additional hyperparameters that need careful tuning, adding to the overall complexity of the training pipeline.

4. Data Augmentation for Multiclass Utterance Study *Xu, B., Qiu, S., Zhang, J., Wang, Y., Shen, X., and de Melo, G., 2020*

This paper investigates data augmentation strategies specifically tailored for multiclass utterance classification in dialogue systems

Advantages: The proposed methods work well across multiple intent categories and show consistent improvements in model robustness when training data is scarce. The paper provides a thorough empirical evaluation across different augmentation strategies, offering useful guidance for practitioners building dialogue systems. The techniques are also relatively straightforward to integrate into existing classification pipelines without major architectural changes.

Disadvantages: The augmentation strategies are designed with dialogue and utterance data in mind, limiting their direct applicability to other text classification tasks such as sentiment analysis or document categorisation. The paper also does not extensively evaluate performance on highly imbalanced datasets, which are common in real-world dialogue systems.

5. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding *Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K., 2019*

BERT introduced a groundbreaking bidirectional transformer architecture pre-trained using masked language modelling and next sentence prediction on large text corpora. This approach allows BERT to develop deep, context-aware word representations that capture both left and right context simultaneously, setting new state-of-the-art results across a wide range of NLP benchmarks including question answering, named entity recognition, and text classification.

Advantages: BERT produces rich, contextual word representations that significantly outperform earlier unidirectional models. Its unified pre-train and fine-tune paradigm simplifies NLP pipeline development by eliminating the need for task-specific architectures. Fine-tuning BERT on downstream tasks is relatively straightforward and has been shown to achieve excellent results across diverse NLP applications.

Disadvantages: BERT's large model size results in slow inference speeds, making it challenging to deploy in real-time or latency-sensitive applications. The model also has a fixed maximum input length of 512 tokens, which limits its applicability to tasks involving very long documents.

6. Learning Data Manipulation for Augmentation and Weighting *Hu, Z., Tan, B., Salakhutdinov, R. R., Mitchell, T. M., and Xing, E. P., 2018*

This earlier version of the data manipulation framework laid the groundwork for the meta-learning approach to data augmentation that the 2022 paper later refined. The model is trained to automatically learn how to augment and weight training samples, establishing the core idea of treating augmentation itself as a learnable component of the training process.

Advantages: This work was pioneering in demonstrating that augmentation policies could be learned rather than manually designed. It showed promising results on standard benchmarks and introduced a conceptually elegant approach to combining augmentation and sample weighting within a single framework. It also opened the door for future research into automated data manipulation strategies in NLP.

Disadvantages: As an earlier version of the framework, it lacks several refinements introduced in later iterations and may underperform compared to more recent approaches on complex or diverse datasets. The meta-learning setup is also inherently sensitive to the choice of validation set used to guide the augmentation learning process, which can introduce instability during training.

7. Data Augmentation for Multiclass Utterance Study *Xu, B., Qiu, S., Zhang, J., Wang, Y., Shen, X., and de Melo, G., 2018*

This earlier study explored augmentation techniques for multiclass utterance classification, investigating how synthetic data generation can compensate for the scarcity of labeled samples across multiple intent classes in conversational systems. It laid useful groundwork for subsequent work in dialogue augmentation and served as an early empirical investigation into this problem space.

Advantages: The study demonstrates clear performance gains from augmentation across multiple text classes and provides valuable early evidence for the utility of data augmentation in intent classification tasks. It offers a systematic comparison of different augmentation strategies, giving researchers a useful reference point for understanding the relative strengths and weaknesses of each approach in a multiclass setting.

Disadvantages: As an earlier work, it does not leverage modern generative models such as GPT or BERT-based generators, which are capable of producing significantly higher-quality augmented samples. The augmentation techniques evaluated are relatively basic compared to what is now available, and the experimental scope is limited to a narrow set of dialogue datasets.

8. Adaptable and Reliable Text Classification Using Large Language Models – Z. Wang, Y. Pang, Y. Lin, X. Zhu, 2024

This paper proposes a novel framework for text classification that leverages large language models (LLMs) to replace traditional machine learning pipelines. The approach focuses on improving adaptability and reliability by utilizing pretrained LLMs that can generalize across different domains with minimal task-specific tuning.

Advantages: The model demonstrates strong adaptability across multiple domains without requiring extensive retraining. It reduces reliance on handcrafted features, saving time and effort in preprocessing. The framework is highly robust when dealing with noisy or unstructured text data. Additionally, it benefits from transfer learning, allowing it to perform well even with limited labeled data.

Disadvantages: The approach depends heavily on large pretrained models, which require significant computational resources. This increases cost and limits accessibility for smaller-scale applications. There may also be latency issues in real-time applications due to model size, and fine-tuning such models can still be complex.

9. A Study on Text Classification in the Age of Large Language Models – P. Trust, R. Minghim, 2024

This study explores how large language models are transforming text classification, particularly in applications such as news classification. It examines the integration of tasks like summarization and translation within classification pipelines.

Advantages: LLMs provide superior contextual awareness, leading to improved classification accuracy compared to traditional models. They can handle multiple NLP tasks simultaneously, increasing efficiency in complex workflows.

Disadvantages: The models often lack interpretability, making it difficult to understand their decision-making process. Performance may vary across specialized or domain-specific datasets. Additionally, reliance on large datasets and pretrained models raises concerns about bias and fairness.

10. Text Classification: How Machine Learning is Revolutionizing Text Categorization – H. Allametal.,2025

This paper provides a comprehensive overview of the evolution of text classification techniques from traditional machine learning methods to modern deep learning and transformer-based approaches. It summarizes key contributions and trends in recent research, emphasizing the growing importance of automation and scalability.

Advantages: Modern approaches significantly improve scalability and allow processing of large volumes of unstructured text efficiently. Automation reduces human intervention, and advanced models achieve high accuracy across various applications.

Disadvantages: Increased model complexity makes implementation and maintenance more challenging. There is a higher risk of bias due to dependence on training data quality. Models may also suffer from performance degradation over time due to data drift, requiring continuous updates and monitoring.

11. A Survey of Text Classification with Transformers: How Wide? How Large? How Long? How Accurate? How Expensive? How Safe? – J. Fields, K. Chovanec, P. Madiraju, 2024

This survey reviews transformer-based approaches to text classification, analyzing various aspects such as model size, accuracy, cost, and safety. It also explores the extension of these models to multimodal data, including text combined with numerical and structured data.

Advantages: Transformer models achieve state-of-the-art accuracy and effectively capture long-range dependencies in text. They are highly flexible and can be adapted to different tasks, including multimodal classification. The models also support large-scale applications and deliver strong performance across benchmarks.

Disadvantages: These models are computationally expensive and require significant energy consumption, raising environmental concerns. Their large size can lead to slower inference times and deployment challenges. Additionally, they may be vulnerable to adversarial inputs and raise safety and ethical concerns in real-world applications.

CHAPTER 3

PROBLEM STATEMENT

3.1 EXISTING SYSTEM

In the existing system, models like CNN, HiSAN, and BERT are trained directly on the available dataset. When there is an imbalance in the data, simple fixes like reweighting or repeating data are used. However, this causes the model to perform poorly on rare or less frequent classes. Some teams use Easy Data Augmentation techniques like synonym replacement, random insertion, swapping, or deleting words to generate more data. But these methods only make small changes to existing sentences—they don't create truly new or meaningful examples. Other systems just repeat the same data many times, which doesn't actually add new information. In short, the current systems struggle with limited data, poor coverage for minority labels, and high effort when trying to collect external data manually.

Disadvantages of the Existing System

- **Data Imbalance Problem:** Models fail to learn rare or minority classes properly.
- **Limited Data Diversity:** Techniques like EDA only modify existing data, not create new ideas.
- **Slow Training:** EDA-based models often need more epochs (time) to learn properly.
- **Poor Accuracy for Rare Labels:** Macro-F1 scores remain low because minority topics are underrepresented.
- **Oversampling Limitation:** Repeating the same data doesn't add new linguistic information.
- **Manual Data Collection is Hard:** Gathering extra data manually is time-consuming and difficult to scale.
- **Low Robustness:** Models trained this way don't perform well on evolving or unseen labels.
- **Lack of Contextual Awareness:** Rule-based augmentation often ignores the broader context of a sentence, resulting in grammatically correct but logically nonsensical data.
- **Domain-Specific Limitations:** General augmentation methods often fail in specialized fields where precise terminology is critical and cannot be easily swapped.
- **Bias Reinforcement:** If the initial small dataset contains inherent biases, simple oversampling or modification will amplify these biases rather than mitigating them.

3.1 PROPOSED SYSTEM

The proposed system is a label-aware generative augmentation model combined with visual analytics to improve text classification performance and make the results more understandable. The system uses ChatGPT to generate new, meaningful data for each label, especially for rare classes in multi-label datasets like Reuters. Instead of just repeating old data, it creates fresh, realistic articles with different lengths and writing styles, giving the model more variety and better learning material. These generated texts are added directly into the training process so that the model learns from both original and newly generated data in every training cycle.

Along with data generation, the system also includes a visual analytics module to display and analyze model performance through interactive graphs, charts, and dashboards. This helps users easily compare results such as accuracy, F1-score, and class-wise performance. It visually represents improvements gained through generative augmentation, making it easier to understand how the model performs on minority and majority classes. The system supports multiple model architectures like CNN, HiSAN, and BERT, and uses PyTorch for implementation. It focuses on improving data diversity, supporting zero-shot labels, and giving users a clear visual understanding of performance improvements through visual analytics.

Advantages of the Proposed System

1. Improves accuracy for rare classes using new label-aware generated data.
2. Adds visual analytics to represent results through graphs and charts for better understanding.
3. Generates diverse and meaningful samples, improving data richness and balance.
4. Faster model training with fewer epochs compared to traditional augmentation.
5. Supports zero-shot learning by generating data for unseen or rare labels.
6. Works with multiple models (CNN, HiSAN, BERT) and is easy to integrate.
7. Eliminates manual data collection, saving time and effort.
8. Provides transparent performance visualization, helping in clear model comparison .

CHAPTER 4

SYSTEM REQUIREMENT

SOFTWARE REQUIREMENT SPECIFICATIONS

4.1 FUNCTIONAL REQUIREMENTS

Below are the functional requirements for the **Label-Aware Generative Augmentation & Text- Classification System**. Each function specifies **Inputs** → **Behavior** → **Outputs**, grouped by modules. UML diagrams (PlantUML) are provided for a Use-Case view and a key Sequence/Activity flow to make the behavior unambiguous.

Data Ingestion & Curation

- **Inputs:** Dataset source (path/URI), label taxonomy, split policy (train/val/test).
- **Behavior:** Validate schema; compute label frequencies and document length stats; persist metadata.
- **Outputs:** Canonical dataset (train/val/test), corpus stats (JSON), label histogram.

Filter/Normalize Text

- **Inputs:** Raw texts, normalization config (lowercasing, punctuation rules, min/max length).
- **Behavior:** Clean, deduplicate, and truncate/pad as configured.
- **Outputs:** Clean text records ready for generation/training.

Label-Conditioned Generation

- **Inputs:** Label set, prompt template(s), per-label budget (e.g., N samples), target length (e.g., 50/150/256 words), style profile.
- **Behavior:** Produce *distinct*, label-aware synthetic texts using the chosen generator; enforce diversity, style, and length constraints; record prompts/responses.
- **Outputs:** EK sample pool with provenance (prompt, seed, generator version, timestamp).

Quality & Safety Filters

- **Inputs:** EK pool, quality rules (language, toxicity, off-topic thresholds), semantic similarity thresholds.
- **Behavior:** Score and filter low-quality or unsafe items; deduplicate near-clones.
- **Outputs:** Approved EK pool; rejection report.

- **Behavior:** Sample EK per minibatch/epoch according to policy; maintain per-label quotas and randomness seeds.
 - **Outputs:** Augmented data stream iterator for the trainer.
- **Provenance Tracking**
- **Inputs:** Prompts, generations, model versions, dataset hashes.
 - **Behavior:** Immutable logging and traceability.
 - **Outputs:** Queryable lineage graphs.

4.1.1 Use Case diagram

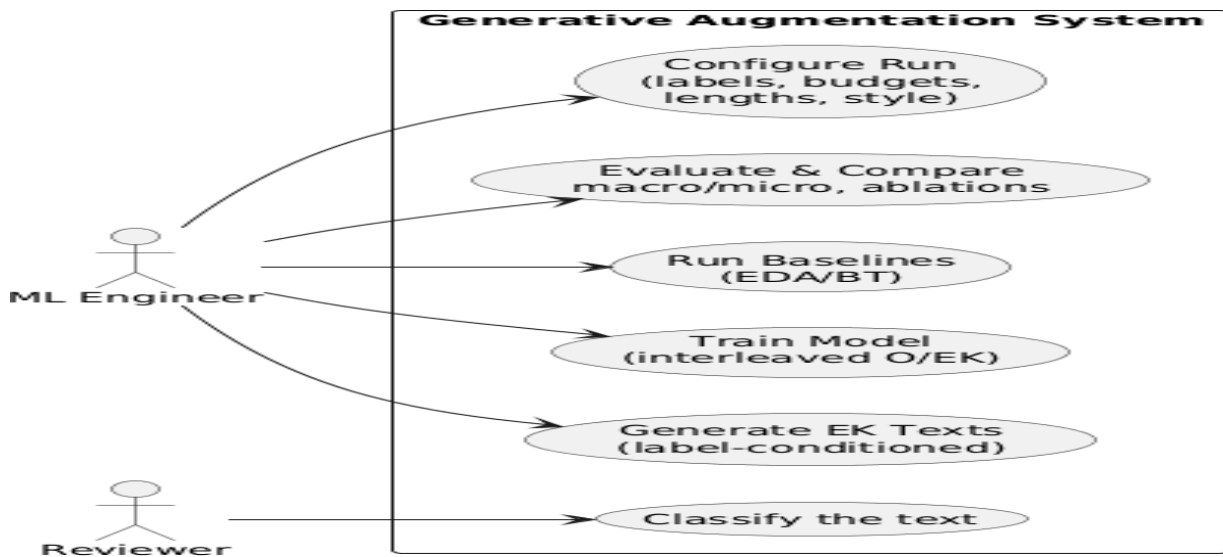


Fig 4.1 Use-Case Diagram

Purpose. Gives a bird’s-eye view of *who* interacts with the system and *what* high-level capabilities they use. It does **not** show order, data formats, or internal algorithms—just the functional surface.

Actors.

- **ML Engineer:** designs runs, triggers generation, training, baselines, evaluations, and exports.
- **CI/CD:** automation that can kick off training/evaluation in pipelines.

System boundary. The rectangle labeled “Generative Augmentation System” encloses the *use cases*

(capabilities) the system provides. Anything outside is a user or external system.

Key use cases.

- **Configure Run:** choose labels, budgets, text length, style, safety thresholds, and model.
- **Generate EK Texts:** create label-conditioned, in-style (e.g., Reuters-style) synthetic samples.

- **Train Model (interleaved O/EK):** run the interleaved schedule (original → EK → original).
- **Run Baselines (EDA/BT):** produce comparison variants (Easy Data Augmentation / Back-Translation).
- **Evaluate & Compare:** compute macro/micro metrics, per-label scores, ablations.
- **include** = mandatory sub-behavior;
- **extend** = optional behavior under a condition.

Why it's useful. It maps directly to your functional requirements and clarifies responsibilities: ML Engineer drives experimentation; Reviewer protects data quality; CI/CD ensures repeatable, automated runs.

4.2 NONFUNCTIONAL REQUIREMENTS

This section defines **verifiable** qualities that constrain the design/implementation of the *Label-Aware Generative Augmentation & Text-Classification System*. Each item includes **Targets/SLOs, Design Implications, and Verification**.

4.2.1 Usability & Operability

Scope: Web UI/CLI used by ML engineers, reviewers, and CI/CD.

- **Run Design Clarity Target/SLO:** Create a valid run (labels, budgets, lengths, model, policies) in ≤ 5 minutes with ≤ 1 validation error on first attempt. Design: Typed forms, inline validation, presets (e.g., 30-distinct/150-words), tooltips, “test prompt” preview. Verification: Usability tests with ≥ 5 users; task completion time/error metrics.
- **Accessibility**

WCAG AA for UI; keyboard-only operation; high-contrast theme. Design: Semantic HTML, ARIA, focus states, contrast checks. Verification: Automated audits (axe), manual checks.

4.2.2 Reliability, Availability & Durability

- **Training Resilience**

Target: No data/epoch loss on single-node failure; restart from last checkpoint with ≤ 5 min lost work.

Design: Epoch- or N-step checkpoints, atomic artifact writes, idempotent resumption.

4.2.3 Performance & Scalability

- **Throughput — Generation**

Target: ≥ 20 EK docs/min/node at 150-word target with filtering enabled.

Design: Async batching, connection pooling, back-pressure; parallel prompt execution.

Verification: Load tests on N nodes; scale curve documented.

- **Training Time Budget**

BERT-base run (20 epochs, interleaved) completes within X hours on $1 \times A100$ with mixed precision; micro-F1 within $\pm 1\%$ of baseline.

FP16/TF32, gradient accumulation, efficient tokenization cache; data loader prefetch.

Verification: Repro runs; wall-clock SLA + metric tolerance.

Target: Linear ($\pm 20\%$) speedup up to K trainer replicas on data-parallel mode.

Design: DDP/FSDP, sharded data loaders, rendezvous server, sticky seeds for determinism.

- **Scalability Strategies Auto-Scaling Triggers:** Explicitly mention what triggers your scaling Model Routing. To save costs, you could add a point about Dynamic Model Routing using a smaller, faster model for simple augmentations and routing complex cases to a larger one.

- **Data Quality during Scaling Drift Monitoring:** As you scale data augmentation, the risk of "topic drift" increases. You could add a point about Automated Evaluation Loops to monitor the similarity between synthetic and original training data to ensure high-quality labels at scale

- **Enhanced Throughput Metrics Beyond the raw "EK docs/min" target, consider adding:**
Latency vs. Batch Size: Document how request latency scales as you increase batch size. This helps identify the "sweet spot" for throughput efficiency without overwhelming the system. token efficiency.

- **Reliability and Cost Optimization Fault Tolerance & Retries:** Scalable systems need to handle failures. Add a point about implementing Exponential Backoff or asynchronous request handling to prevent workflow blocking during API or hardware timeouts.

4.3 Hardware Requirements

Table 4.1 : Hardware Requirements

Component/Role	Recommended Specification	Minimum Specification	Purpose
Developer Workstation(ML Engineer)	CPU: 8C/16T (Ryzen 7 / i7) RAM: 32 GB Storage: 1 TB NVMe SSD GPU: RTX 4070 Network: 200 Mbps	CPU: 4C/8T RAM: 16 GB Storage: 512 GB SSD	Local model development and testing
Training GPU Node (Model Training)	CPU: 24C+ RAM: 128 GB GPU: 1× A100 40/80 GB or 2× RTX 6000/4090 Storage: 2 TB NVMe Network: 10 GbE	CPU: 12C RAM: 64 GB GPU: 1× RTX 3080 10 GB Storage: 1 TB	Model training and optimization
EK Generation Node(LLM Client)	CPU: 16C RAM: 64 GB Storage: 1 TB NVMe Optional small GPU	CPU: 8C RAM: 32 GB Storage: 512 GB SSD	LLM text generation / inference
Augmentation / Run Manager	vCPU: 4–8 RAM: 16–32 GB Storage: 200 GB SSD	CPU: 2 RAM: 8 GB	Manages API and control operations
PostgreSQL Metadata DB	vCPU: 8 RAM: 32 GB Storage: 500 GB NVMe	vCPU: 4 RAM: 16 GB Storage: 200 GB SSD	Stores metadata, indices, checkpoints
Object Storage (MinIO / S3 / GCS)	10–20 TB usable (S3-compatible API)	2–4 TB	Stores datasets, checkpoints, and artifacts

4.4 SOFTWARE REQUIREMENTS

Table 4.2 : Software requirements

Layer	Component	Version/Notes	Purpose
OS	Linux (Ubuntu 22.04 LTS / RHEL)	Latest LTS	Stability & long-term support
GPU Drivers	NVIDIA Driver + CUDA + cuDNN	CUDA 12.x + cuDNN 9.x (match PyTorch)	GPU acceleration
Runtime	Python	3.10 – 3.11	Core language
ML Frameworks	PyTorch, TorchMetrics	2.3+ (latest with AMP)	Model training & metrics
NLP / Tokenization	transformers, tokenizers, datasets	Latest stable	Text processing
Classical ML	scikit-learn, pandas, numpy	Latest stable	Pre/Post processing
Text Utils (Optional)	spaCy, NLTK, regex Optional	Latest stable	Normalization & filtering
Serving	FastAPI + Uvicorn/Gunicorn	Latest	Model API endpoints
Orchestration	Airflow or Prefect	Latest LTS	Scheduling & workflow
DB	PostgreSQL	14 – 16	Data storage
LLM Access	OpenAI / Azure / Open-source LLMs	API SDKs pinned	Text generation (API-based)
CI/CD GitHub	Actions / GitLab CI / Jenkins	Latest	Build, test, deploy

CHAPTER 5

SYSTEM ANALYSIS AND DESIGN

5.1 SYSTEM ANALYSIS

Below is the structured analysis that was performed to plan the design of the Label-Aware Generative Augmentation & Text-Classification System.

1. Problem Context & Stakeholders

Context. Multi-label text classifiers (e.g., on Reuters topics) underperform on long-tail labels due to data scarcity and distribution shift. Traditional fixes (oversampling, EDA) add quantity but limited semantic coverage.

Primary stakeholders. ML Engineers (experimentation), Reviewers/Domain Experts (content quality & safety), Product/Research Leads (results & reproducibility), MLOps/Platform (deployment & cost), and Compliance/Security (governance).

Business goal. Raise macro-F1 (minority labels) with minimal data-collection cost and keep training/runtime budgets predictable.

2. Objectives & Success Metrics

- **Effectiveness:** Increase macro-F1 by $\geq +10-15$ points over non-augmented baseline while keeping micro-F1 within $\pm 1-2\%$.
- **Efficiency:** Reach target accuracy in fewer epochs than EDA baseline (e.g., 30–50% reduction).
- **Coverage:** Enable zero-shot/rare labels via label-conditioned generation.
- **Reproducibility & Safety:** 100% EK items with provenance; 0 unsafe items admitted to training.

3. Current-State Analysis

- **Data:** Imbalanced multi-label corpus; majority classes dominate gradients; short documents common.
- **Pipelines:** Standard CNN/HiSAN/BERT training; optional oversampling/EDA; limited ablations; slow convergence for attention models on minority classes.
- **Pain points:** Low minority recall, convergence time, inability to address unseen labels, manual data collection overhead.

4. Scope & Decomposition

- **Data Ingestion & Curation** → canonical splits, stats, normalization.
- **EK Generation** → label-conditioned prompts; budgets; length tiers (50/150/256).
- **Quality & Safety Filtering** → toxicity/off-topic/PII checks; dedup & similarity bounds
- **Provenance & Artifact Store** → prompts, seeds, model/commit versions, metrics bundle.
- **Reviewer Console & API** → approvals, overrides, run management.

Conceptual Data Flows (Analysis View)

- **Data Flow Diagram L0 (textual):** Actors: ML Engineer

Processes: (P1) Ingest & Profile → (P2) Generate EK → (P3) Filter & Approve → (P4) Mix & Train
→ (P5) Evaluate & Report

Stores: D1 Corpus, D2 EK Pool (Approved/Rejected), D3 Checkpoints/Artifacts, D4
Metrics/Lineage

Flows:

- Corpus → P1 → stats → P2;
- P2 → candidates → P3 → (Approved EK → D2, Rejected EK → log);
- D1 + D2 → P4 → models → D3;
- P4 → metrics → P5 → dashboards/exports → stakeholders.

5. Capacity & Sizing Estimates

- **EK throughput target:** ≥ 20 approved texts/min/node @ 150 words with filters.
- **Storage:** Dataset + EK + checkpoints $\approx 5\text{--}10\times$ raw corpus size over a quarter.
- **Training time:** BERT-base 15–20 epochs on $1\times 24\text{--}40$ GB GPU; interleaving adds modest I/O overhead mitigated by prefetch and NVMe.

6. Test Strategy

- **Ablations:** Distinct vs. repeated EK; 50 vs. 150 vs. 256 words; EK vs. EDA vs. None.
- **Statistical tests:** Paired comparisons across seeds; confidence intervals on macro-F1.
- **Safety tests:** Seeded batches with known edge prompts to verify filters/quarantine.
- **Repro runs:** Re-execute same run bundle; assert $\leq 0.5\%$ metric drift.

7.Expected Outcomes & Decision Gates

- **Go** if macro-F1 improves by ≥ 10 points on targeted labels, convergence cost drops $\geq 30\%$, and safety/repro checks pass.
- **Refine** prompts/filters or EK ratios if gains are inconsistent across labels.
- **Stop** if EK harms micro-F1 materially or fails safety/reproducibility gates.

5.2 HIGHLEVEL DESIGN

5.2.1 System Architecture

The system is organized as a set of purpose-built subsystems that work together to generate label- aware external-knowledge (EK) texts, mix them with the original corpus, train text classifiers (CNN/HiSAN/BERT) using an interleaved schedule, and evaluate/report results with full provenance and safety controls.

5.2.1.1 Layers and roles

At a high level there are four layers:

- **Experience & Control Layer** — a web Config UI and Run Manager API (with SSO/RBAC) where ML Engineers create runs (labels, budgets, EK lengths, model/epochs) and Reviewers approve or reject generated EK. CI/CD can trigger or schedule runs.
- **Data & EK Layer** — Ingestion & Curation prepares the canonical dataset (splits, normalization, stats). EK Generation & Filtering builds prompts, calls an LLM (hosted or on-prem), enforces safety (toxicity/PII/lang) and diversity checks, and records provenance. Approved items form the EK Pool.
- **Training & Evaluation Layer** — the Augmentation Manager mixes original and EK samples into interleaved minibatches for the Trainer (PyTorch; AMP; optional DDP/FSDP). Evaluation & Reporting computes macro/micro P/R/F1, per-label scorecards, ablations, and exports a reproducible bundle.
- **State & Observability Layer** — PostgreSQL stores configs, lineage, and metrics indices; an Object Store (S3/MinIO) holds EK texts, checkpoints, and reports; Redis/Queue coordinates jobs and caches; Prometheus/Grafana/Loki expose metrics, dashboards, and logs.

5.2.1.2 Core subsystems (what each does)

- Ingestion & Curation profiles the corpus (label histogram, length distribution), validates schema, deduplicates, and produces train/val/test splits plus a stats JSON used to pick EK length tiers (e.g., 50/150/256 words).
- EK Generation & Filtering constructs label-conditioned prompts , generates candidates via LLM, enforces style/length bounds, computes semantic similarity to ensure distinctness, applies safety filters, and offers a Reviewer screen for approvals. Every EK item is stamped with {prompt_id, seed, model_version, scores, decision}.
- Augmentation Manager implements the policy that truly differentiates the system: interleaving original and EK batches with configurable ratios and per-label For each label in the taxonomy ChatGPT is prompted to generate short news articles in domain style.
- Label-conditioned text generation. For each label in the taxonomy ChatGPT is prompted to generate short news articles in domain style. The method explicitly varies the number, diversity, and length of generated samples to test whether *new*, *distinct* examples outperform duplicated ones and whether *longer* articles carry more useful signal than very short snippets.

5.2.2 External environment & human interaction

- **Humans:** ML Engineers design and launch experiments; Reviewers curate EK quality/safety; Leads consume dashboards and exported reports; SREs manage clusters, storage, and backups.
- **External systems:** an LLM provider (if not on-prem) for generation; an Identity Provider (OIDC/SAML) for SSO; and optional CI/CD to schedule baselines or nightly evaluations.

5.2.3 Scalability, reliability, and security (built-in qualities)

- **Scale:** Training scales horizontally with DDP/FSDP across GPU workers; EK-Gen scales out independently; queues add back-pressure so training can continue with cached EK if generation lags.
- **Resilience:** Periodic checkpoints and idempotent resumption cap work loss object storage is versioned Postgres has PITR backups health probes and autoscaling keep the control plane available.
- **Security & governance:** RBAC across UI/API secrets in a KMS/secret manager EK has provenance retention/lifecycle policies purge stale artifacts model cards and audit logs capture and what was trained

5.2.4 Data Flow Diagram

DFD-Level-0: DFD Level-0 (Context Diagram)

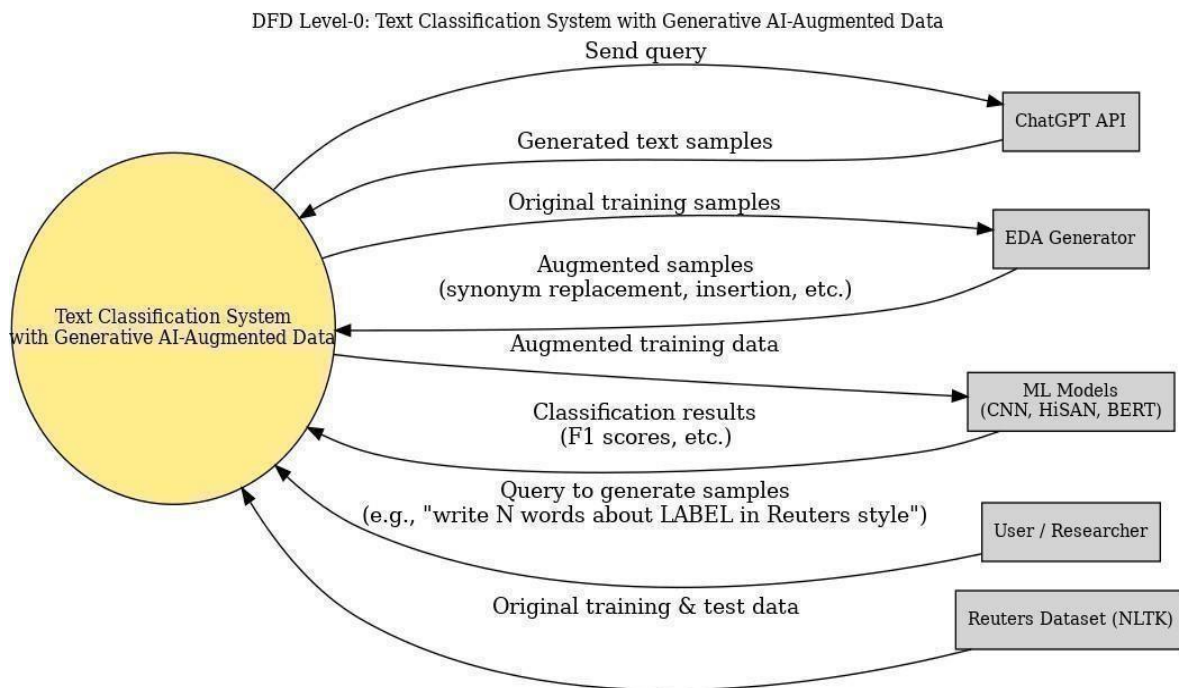


Fig 5.1 Data Flow Diagram Level-0(Context) Diagram

Objective: Show the system as a single process and its interactions with external entities.

Process:

- Text Classification System with Generative AI-Augmented Data External Entities:

- ChatGPT API
- EDA Generator
- Reuters Dataset (NLTK)
- ML Models (CNN, HiSAN, BERT)

Data Flows:

- User → System: Query for generating samples (e.g., "write an article with N words about LABEL in Reuters news format")
- System → ChatGPT API: Send query.
- ChatGPT API → System: Return generated text samples.
- System → EDA Generator: Original training samples.
- EDA Generator → System: Augmented samples via synonym replacement, insertion, etc.
- Reuters Dataset → System: Original training and test data.
- System → ML Models: Augmented training data
- ML Models → System: Classification results (F1 scores, etc.)

Data Flow Diagram Level-1 (High-Level Process)

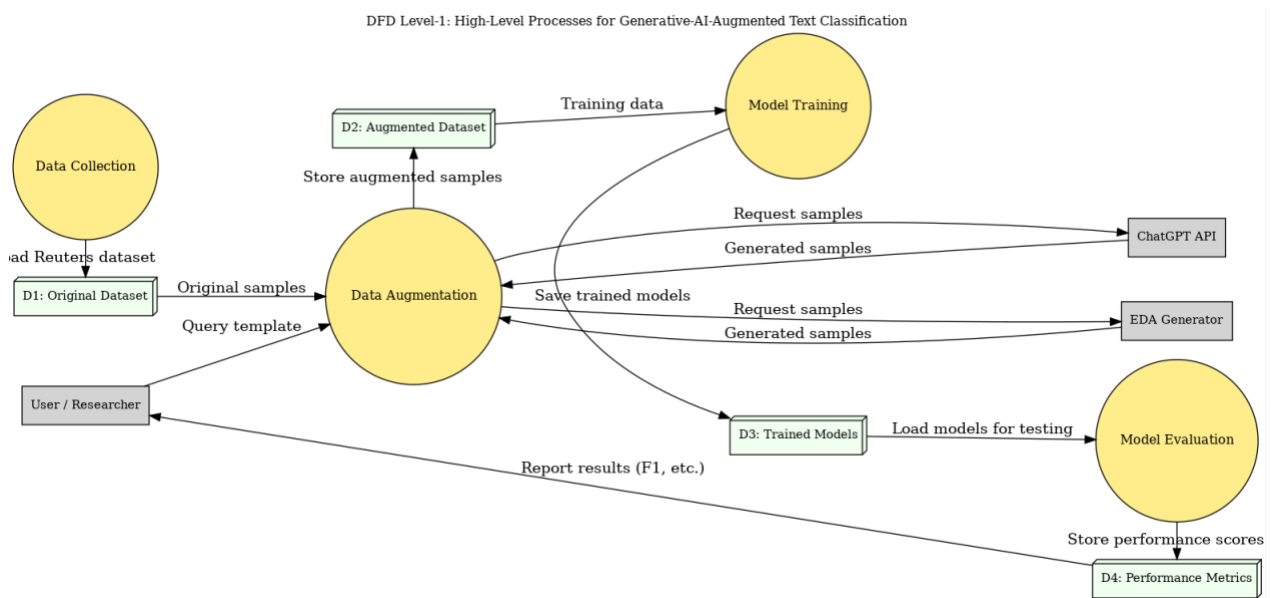


Fig 5.2 Data Flow Diagram Level-1(High Level Process)

Diagram Data Flow Diagram Level-1 (High-Level Process)

Objective: Break down the main process into key sub-processes.

Processes:

1. Data Collection
2. Data Augmentation
3. Model Training
4. Model

Evaluation Data

Stores:

- D1: Original Dataset
- D2: Augmented Dataset
- D3: Trained Models
- D4: Performance Metrics

Data Flows:

- User → Data Augmentation: Query template

Data Augmentation → ChatGPT API / EDA Generator: Request samples

- ChatGPT API / EDA Generator → Data Augmentation: Generated samples
- DataCollection → D1: Load Reuters dataset
- D1 → Data Augmentation: Original samples
- Data Augmentation → D2: Store augmented samples
- D2 → Model Training: Training data
- Model Training → D3: Save trained models
- D3 → Model Evaluation: Load models for testing
- Model Evaluation → D4: Store performance scores
- D4 → User: Report results (F1 scores, etc.)

Data Flow Diagram Level-2 (Detailed Process for Data Augmentation)

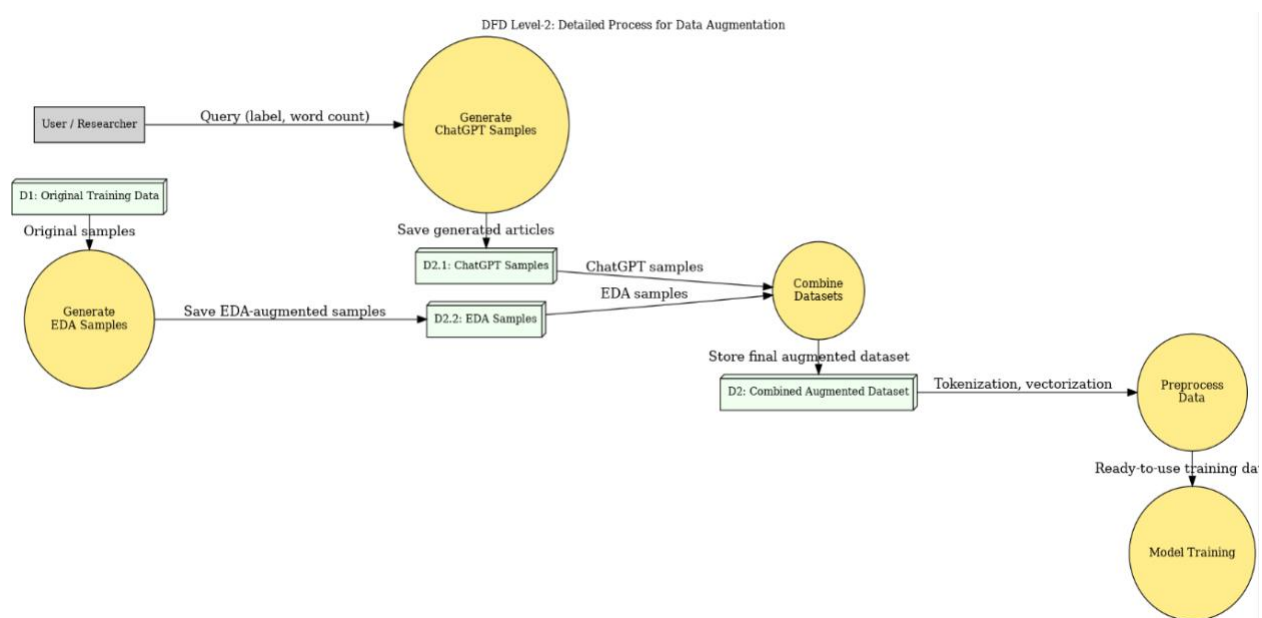


Fig 5.3 Data Flow Diagram Level-2(Data Augmentation) Diagram

Objective: Detail the Data Augmentation process from Level-1.

Sub-Processes:

1. Generate ChatGPT Samples
2. Generate EDA Samples
3. Combine Datasets
4. Preprocess Data

Data Flows:

- User → Generate ChatGPT Samples: Query (label, word count)
- Generate ChatGPT Samples → D2.1: Save generated articles
- D1 → Generate EDA Samples: Original samples
- Generate EDA Samples → D2.2: Save EDA-augmented samples
- D2.1, D2.2 → Combine Datasets: Merge external and EDA samples
- Combine Datasets → D2: Store final augmented dataset
- D2 → Preprocess Data: Tokenization, vectorization
- Preprocess Data → Model Training: Ready-to-use training data

5.4 DB Design:

Use data flow diagrams to explain the flow of data in sub systems and modules

5.4.1 DB table structure:

Should specify the tables with their various fields, indexes and relations with an ER diagram. Show the normalization done on the tables. Highlight the maximum number of records that the DB will hold with the design for testing. Expanded Workflow Points Prompt Engineering Strategy: Define the prompts used for the Generative AI to ensure high-quality synthetic data. Quality Filtering: Implement a "Consistency Check" where a pre-trained model validates if the augmented text still aligns with its original label . Use Cosine Similarity or T-SNE plots to ensure the generated data provides new semantic variations rather than just repeating original patterns

.Database Design Details Normalization, Mention that the database follows 3rd Normal Form (3NF) to ensure that data augmentation parameters (like temperature or top-p) are stored in a separate config table to reduce redundancy.

5.4.1.2 Entity Relationship Diagram

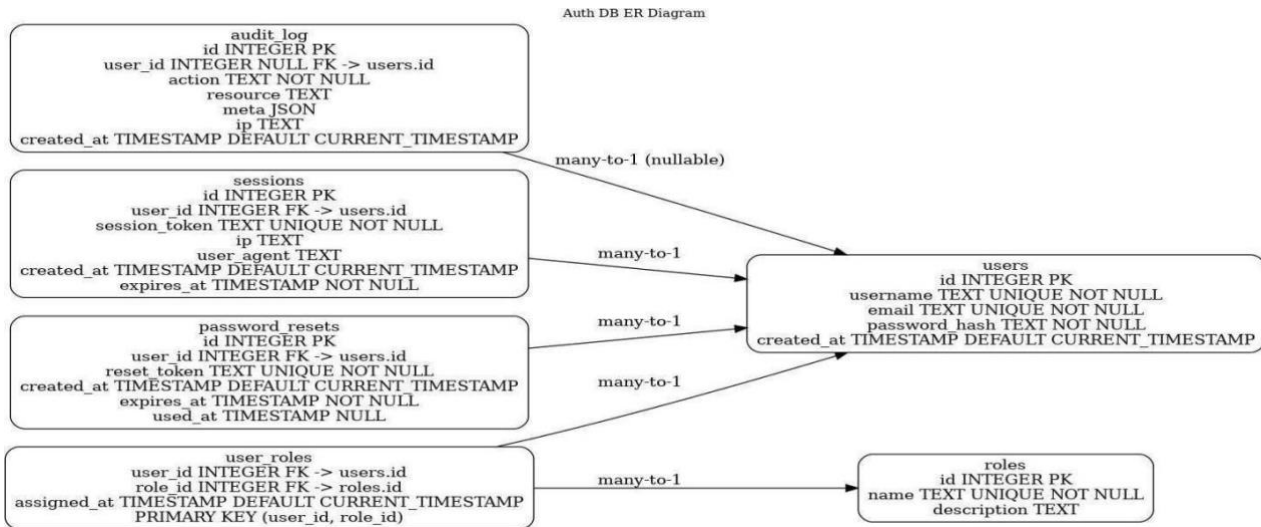


Fig 5.4 Entity Relationship Diagram

The ER (Entity–Relationship) diagram models the authentication database used by the system. It contains the following entities: users, roles, sessions, password_resets, audit_log, and user_roles (assigned roles).

users: Primary entity that stores user account information. Important attributes include id (primary key), username (unique), email (unique), password_hash, and created_at (timestamp). Each user record represents one account in the system.

user_roles (assigned roles): Associative table that links users and roles. It contains user_id (foreign key → users.id) and role_id (foreign key → roles.id). The composite primary key (user_id, role_id) prevents duplicate role assignments. This table implements the many-to-many relationship between users and roles via two many-to-one links (many user_roles → one users; many user_roles → one roles).

sessions: Stores active or historical login sessions. Key attributes include id (primary key), user_id (foreign key → users.id), session_token (unique), ip, user_agent, created_at, and expires_at. Each session record is associated with exactly one user (many sessions → one user).

password_resets: Contains password reset requests and tokens. Attributes include id (primary key), user_id (foreign key → users.id), reset_token (unique), created_at, expires_at, and used_at. Each reset entry links to the requesting user.

5.4.2 Low Level Design

The system consists of four main modules:

- D1: Original Training Data
- D2.1: ChatGPT Samples
- D2.2: EDA Samples
- D2: Combined Augmented Dataset
- Data Augmentation Module
- Model Training Module
- Evaluation Module
- Configuration Manager

5.4.2.1 Module Detailed Descriptions

1.1 Purpose and Responsibilities

The Data Loader Module is responsible for loading, parsing, and preprocessing the raw text dataset. It handles the initial data acquisition from the Reuters corpus through NLTK and prepares the data for both training and augmentation processes.

Core Components

Reuters Data Loader Class

- **Primary Function:** Loads and partitions the Reuters dataset
- **Key Methods:**
 - `load_reuters_corpus()`: Downloads and loads the dataset from NLTK
 - `split_dataset()`: Creates training (90%), validation (10%), and test splits
 - `extract_labels()`: Processes multi-label annotations for 90 categories

Text Preprocessor Class

- **Primary Function:** Standardizes text processing across all data sources
- **Processing Pipeline:**
 - **Tokenization:** Converts text to tokens using NLTK tokenizer
 - **Lowercasing:** Normalizes all text to lowercase
 - **Stopword Removal:** Removes common English stopwords

5.5 APPLICATIONS

- Healthcare & Clinical Reports :- Handling rare disease classification where real samples are scarce.
- Legal & Government Documents:-Augmenting under-represented categories in legal corpora.
- News & Media Monitoring :- Improving topic classification with balanced datasets.
- Multilingual NLP Systems :- Extending coverage to low-resource or unseen languages.
- Financial & Security Reports :-Enhancing fraud detection or sentiment analysis for rare cases.
- Customer Support Automation:- Training chatbots to recognize rare or complex technical issues by augmenting sparse ticket data.
- Cybersecurity & Threat Intelligence:- Classifying evolving phishing tactics or zero-day exploit descriptions that lack extensive historical logs.
- Content Moderation: -Improving the detection of nuanced hate speech or "coded" harmful language that changes frequently to bypass filters.
- E-commerce Product Categorization: -Mapping new or unique product lines to standard taxonomies based on limited initial descriptions.
- Human Resources (HR Tech): -Sorting resumes for highly specialized roles where the training set of "ideal" candidates is very small.
- Sentiment Analysis for Niche Brands:- Training models to understand the specific slang or context of a small subculture or hobbyist community.
- Disaster Management: Quickly classifying social media calls for help during emergencies to prioritize rescue efforts based on limited early reports.
- Academic & Scientific Research:- Automating the tagging of interdisciplinary papers that don't fit neatly into established, data-rich categories.
- Insurance Claim Processing:- Identifying specific types of rare or complex insurance fraud patterns within textual claim descriptions.

CHAPTER 6

IMPLEMENTATIONS

The project “Enhancing Text Classification Models with Generative AI-aided Data Augmentation” is implemented using Python, PyTorch, Hugging Face Transformers, NLTK, and Flask. The system consists of two main phases: (1) training with generative data augmentation and (2) deployment for real-time classification. In the training phase, the Reuters multi-label dataset is used. Each document contains multiple topic labels, making it a multi-label classification problem. The dataset is preprocessed by converting text into string format and encoding labels using MultiLabelBinarizer, which transforms multiple labels into binary vectors. The dataset is then split into training, validation, and test sets.

Module 1: Data Collection (Reuters Multi-Label Dataset)

- Load Reuters documents and their associated topic categories.
- Extract raw text and label lists for each document.
- Maintain a list of all unique labels (topics) present in the dataset.

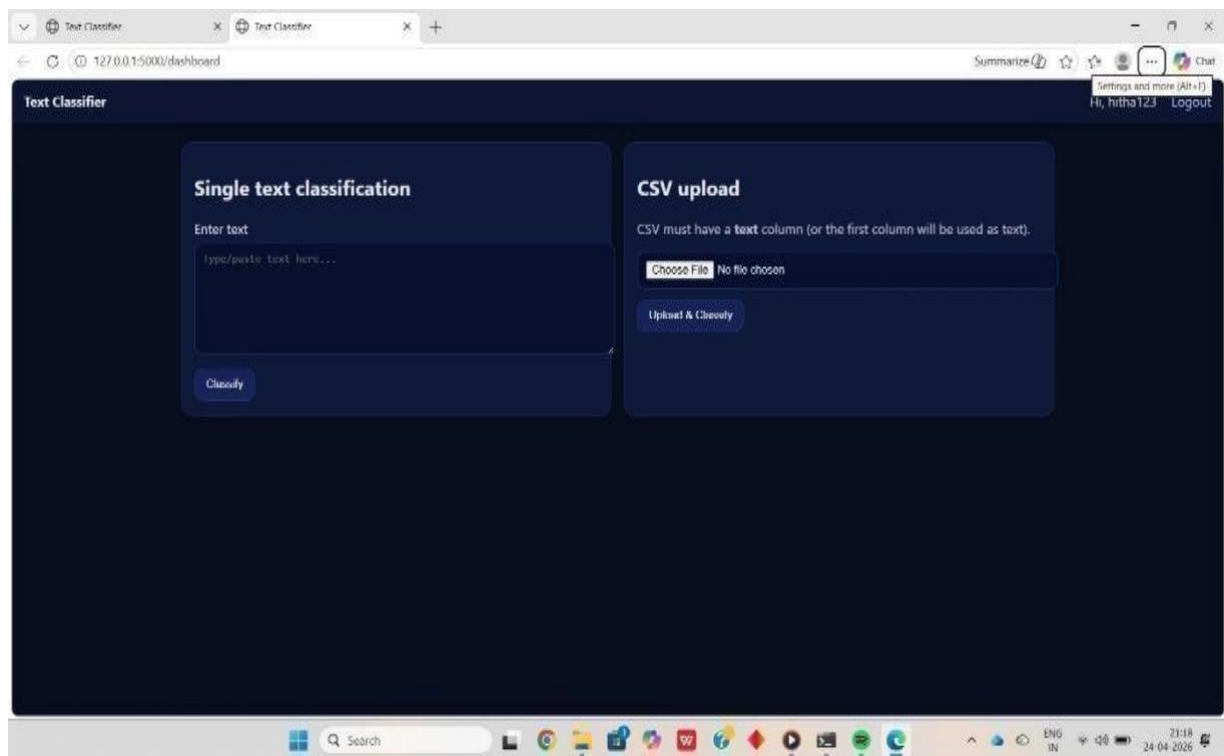


Fig 6.1 User input result

Module 2: Data Preprocessing & Label Encoding

- Clean and normalize text (convert to string, remove empty/null samples).
- Filter documents based on label-count constraints (avoid extreme cases).
- Convert multi-label categories into binary vectors using MultiLabelBinarizer.
- Split the dataset into Train / Validation / Test.

Module 3: Generative AI-Aided Data Augmentation

- Identify training samples belonging to minority/long-tail labels.
- Generate synthetic label-aware training samples using LLM prompting strategies:
 - Paraphrasing (same meaning, different wording)
 - Expansion (add relevant details)
 - Summarization (shorter form preserving meaning)
 - Back-translation (translate and back to add semantic diversity)
- Control generation length (e.g., 50/150/256 words) to study richness impact.
- Filter low-quality outputs (duplicates, too short, irrelevant).
- Merge synthetic samples with original training data to form the augmented dataset.

Module 4: Tokenization & Dataset Loader

- Use BERT tokenizer (bert-base-uncased) to convert text into:
 - input_ids
 - attention_mask
- Apply padding and truncation to a fixed maximum length (e.g., 256).
- Implement a PyTorch Dataset/DataLoader to feed batches into models.

Module 5: Model Building (Multi-Label Classifier)

- Implement multi-label classifiers (core focus on BERT):
 - BERT → Dropout → Linear Layer → Sigmoid
- Sigmoid is used to allow multiple labels per input (multi-label output).
- Optionally train and compare with CNN and HiSAN models for analysis.

Module 6: Training & Model Selection

- Train the model using:
 - Binary Cross Entropy (BCE) loss
 - AdamW optimizer
- Perform validation after each epoch.
- Save the best model based on validation loss as best_model.pt.
- Store training curves/metrics for analysis.

Module 7: Evaluation & Comparative Analysis

- Run testing on the held-out test set.
- Convert probabilities to predicted labels using a threshold (e.g., 0.5).
- Compute evaluation metrics:
 - Macro Precision/Recall/F1 (important for minority labels)
 - Micro Precision/Recall/F1
- Compare results across:
 - No augmentation
 - Oversampling
 - EDA
 - Back-translation
 - Proposed generative augmentation

Module 8: Deployment (Flask Web Application)

- Build a Flask application with:
 - User Registration/Login (Flask-Login)
 - SQLite database for user storage
- Load trained model (best_model.pt) and label encoder (mlb.pkl) once at startup.
- Provide two prediction modes:
 - Single Text Prediction (shows top-k probabilities + predicted labels)
 - Batch Prediction via file upload (counts labels + preview results)

Module 9: Result Display & User Interaction

- Display predictions in web UI:
 - Predicted labels above threshold
 - Top-k label confidence scores
- For batch classification:
 - Show first 10 predictions as preview
 - Display top label frequency summary (chart-ready output)

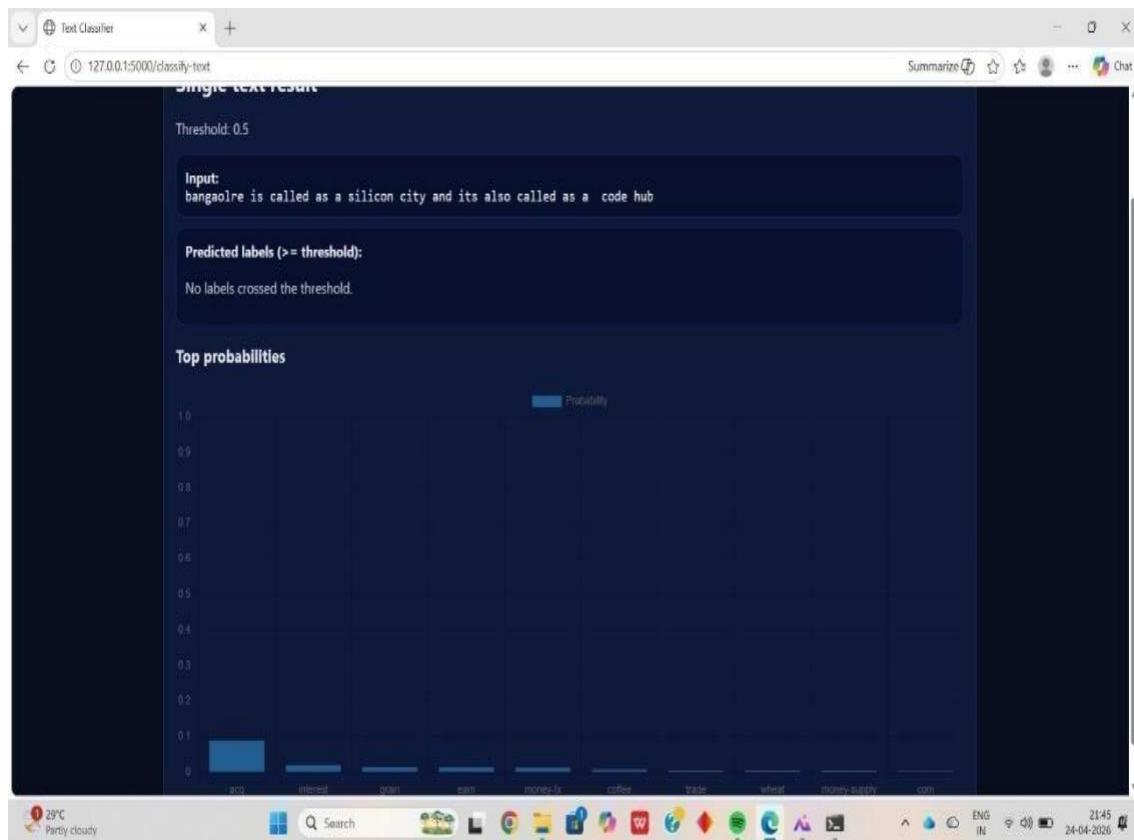


Fig 6.2 Output

In the given figure input text, the model processes it and predicts probabilities for each label. We set a threshold of 0.5, so only labels above that are selected. In this case, no label crossed the threshold, so no final label is shown, but we can still see the probability distribution in the graph.

CONCLUSION

In this project, we successfully developed a multi-label text classification system using the BERT model. We improved the performance of the model by applying data augmentation techniques like paraphrasing and summarization, which helped in handling less frequent labels. The data was properly cleaned, processed, and used to train the model effectively. We evaluated the model using appropriate metrics to ensure good performance. Finally, we deployed the system using a Flask web application, allowing users to give input text and get predictions in real time. Overall, the project provides an efficient and practical solution for classifying text into multiple categories. Our system starts with the user giving input through a Flask-based web interface. The input text is then sent to the processing layer, where it is cleaned and prepared, and data augmentation techniques are applied to improve the quality of data. After that, the processed text is converted into a format suitable for the model using tokenization, and it is passed to the BERT model, which analyzes the text and generates probability scores for each category. Finally, in the output layer, the system selects the most relevant labels using a threshold or Top-K method and displays the results to the user, while optionally storing them in a database.

FUTURE ENHANCEMENT

The proposed text classification system using generative AI-aided data augmentation can be further enhanced in several ways to improve its performance and real-world applicability. Firstly, more advanced data augmentation techniques can be incorporated by leveraging improved prompt engineering and large language models to generate high-quality, context-aware synthetic data. Additionally, a human-in-the-loop validation mechanism can be introduced to filter out noisy or incorrect generated samples, ensuring better training data quality. The system can also be evaluated on multiple real-world datasets from different domains such as news articles, customer reviews, and social media data to test its generalization capability. Furthermore, ensemble learning techniques combining models like CNN, HiSAN, and BERT can be implemented to achieve higher classification accuracy and robustness. Another important enhancement is the integration of explainable AI techniques such as LIME or SHAP, which will help in understanding and interpreting model predictions. The system can also be extended to support multilingual text classification using transformer-based multilingual models, thereby increasing its usability across different languages. Finally, the project can be deployed as a complete web-based application with features like batch processing, visualization dashboards, and user feedback integration. Incorporating active learning strategies can further improve the system by allowing it to continuously learn from new and uncertain data samples, making it more adaptive and efficient over time.

REFERENCES

- [1] Z. Wang, Y. Pang, Y. Lin, and X. Zhu, "Adaptable and reliable text classification using large language models," IEEE International Conference, 2024.
- [2] P. Trust and R. Minghim, "A study on text classification in the age of large language models," Machine Learning and Knowledge Extraction, 2024.
- [3] S. P. K. Veeranki, A. Abdulnazar, D. Kramer, et al., "Multi-label text classification via secondary use of large clinical real-world data sets," Scientific Reports, 2024.
- [4] S. Molitorys, B. Pilot, and K. Smyrak, "Text Classification," CEUR Workshop Proceedings (IVUS), 2024.
- [5] E. De Santis, A. Martino, F. Ronci, et al., "From bag-of-words to transformers: A comparative study for text classification in healthcare discussions in social media," IEEE Transactions, 2024.
- [6] L. Xiao, Q. Li, Q. Ma, J. Shen, Y. Yang, and D. Li, "Text classification algorithm of tourist attractions subcategories with modified TF-IDF and Word2Vec," PLOS ONE, 2024.
- [7] H. Zhao, H. Chen, T. A. Ruggles, Y. Feng, D. Singh, et al., "Improving text classification with large language model-based data augmentation," Electronics, 2024.
- [8] M. Cesarini, L. Malandri, F. Pallucchini, A. Seveso, et al., "Explainable AI for text classification: Lessons from a comprehensive evaluation of post hoc methods," Springer (Cognitive Studies), 2024.
- [9] Guo, J., Wang, M., Yin, H., Song, B., Chi, Y., Yu, F. R., & Yuen, C. (2024). Large language models and artificial intelligence generated content technologies meet communication networks. *IEEE Internet of Things Journal*, 12(2), 1529-1553.
- [10] Andreoni, M., Lunardi, W. T., Lawton, G., & Thakkar, S. (2024). Enhancing autonomous system security and resilience with generative AI: A comprehensive survey. *IEEE Access*, 12, 109470-109493.

- [11] Wang, N., Chiong, R., Kamil, R., Zhang, W., Al-Haddad, S. A. R., & Ibrahim, N. (2024). Depression detection using speech audio and text: A comprehensive review focusing on deep learning methods. *Authorea Preprints*.
- [12] Lifelo, Z., Ding, J., Ning, H., & Dhelim, S. (2024). Artificial intelligence-enabled metaverse for sustainable smart cities: Technologies, applications, challenges, and future directions. *Electronics*, 13(24), 4874.
- [13] Gupta, M., & Mishra, A. (2024). A systematic review of deep learning based image segmentation to detect polyp. *The Artificial Intelligence Review*, 57(1), 7.
- [14] Gangwal, A., Ansari, A., Ahmad, I., Azad, A. K., Kumarasamy, V., Subramaniyan, V., & Wong, L. S. (2024). Generative artificial intelligence in drug discovery: basic framework, recent advances, challenges, and opportunities. *Frontiers in pharmacology*, 15, 1331062.
- [15] Andreoni, M., Lunardi, W. T., Lawton, G., & Thakkar, S. (2024). Enhancing autonomous system security and resilience with generative AI: A comprehensive survey. *IEEE Access*, 12, 109470-109493.
- [16] Kondylakis, H., Osuala, R., Puig-Bosch, X., Lazrak, N., Diaz, O., Kushibar, K., ... & Lekadir, K. (2025). A review of methods for trustworthy AI in medical imaging: The FUTURE-AI Guidelines. *IEEE Journal of Biomedical and Health Informatics*.
- [17] S. Liu, S. Liu, L. Sha, Z. Zeng, et al., "Annotation guideline-based knowledge augmentation: Towards enhancing large language models for educational text classification," *IEEE Transactions*, 2025.
- [18] Wang, Y., Wang, L., & Siau, K. L. (2025). Human-centered interaction in virtual worlds: A new era of generative artificial intelligence and metaverse. *International Journal of Human-Computer Interaction*, 41(2), 1459-1501.
- [19] Luong, N. C., Hai, N. D., Van Le, D., Nguyen, H. T., Vu, T. H., Huynh-The, T., ... & Pham, Q. V. (2025). Diffusion models for future networks and communications: A comprehensive survey. *arXiv preprint arXiv:2508.01586*.

[19] H. Sakai and S. S. Lam, "Large language models for health care text classification: Systematic review," JMIR AI, 2026.

[20] Xu, L. C., Zhu, D. X., Tang, M. J., Liu, T. T., Liu, Y. Z., Ma, C. H., ... & Hong, X. (2026). Transforming Molecular Synthesis With Large Language Models. *Chemistry—A European Journal*, e71074.